

Title: Design a YACC-Based Arithmetic Expression Syntax Analyzer

1. Objective

The objective of this assignment is to design and implement a syntax analyzer using **YACC (Yet Another Compiler Compiler)** for arithmetic expressions. The parser should recognize valid arithmetic expressions, evaluate them according to operator precedence rules, and detect syntax errors.

2. Learning Outcomes

After completing this assignment, students will be able to:

- Understand the role of YACC in compiler construction
- Design context-free grammars for arithmetic expressions
- Implement syntax analysis using YACC
- Integrate FLEX/LEX with YACC
- Handle operator precedence and associativity
- Generate and test parsers

3. Problem Statement

Develop a YACC-based syntax analyzer capable of:

- Recognizing valid arithmetic expressions
- Supporting arithmetic operators
- Evaluating arithmetic expressions
- Detecting syntax errors

Students must complete the parser to support:

- Addition +
- Subtraction -
- Multiplication *
- Division /
- Exponentiation ^
- Parentheses ()

4. Software Requirements

Students may use either **Windows** or **Linux** environment.

Required software/tools:

- FLEX or LEX
- YACC or BISON
- GCC Compiler / MinGW GCC

5. Theory Overview

5.1 What is YACC?

YACC (Yet Another Compiler Compiler) is a parser generator used to generate syntax analyzers from context-free grammar specifications. YACC is widely used in compiler construction for implementing parsers.

5.2 Structure of a YACC Program

A YACC program generally consists of three sections:

Definitions

%%

Grammar Rules

%%

User Programs

5.3 Role of FLEX/LEX

LEX or FLEX performs lexical analysis. It scans the input stream and converts characters into tokens for YACC.

Example Tokens

Input	Token
25	NUMBER
+	PLUS
*	MUL
^	POWER

6. Expression Grammar

The following context-free grammar can be used for arithmetic expressions:

expr $\rightarrow$ expr + term

expr $\rightarrow$ expr - term

expr $\rightarrow$ term

term $\rightarrow$ term * factor

term $\rightarrow$ term / factor

term $\rightarrow$ factor

factor $\rightarrow$ factor ^ primary

factor $\rightarrow$ primary

primary $\rightarrow$ (expr)

primary $\rightarrow$ NUMBER

7. Partial FLEX Program (lexer.l)

Students must complete the lexical rules for all operators.

```
%{
#include "y.tab.h"
#include <stdlib.h>
%}

%%

[0-9]+    {
            yylval = atoi(yytext);
            return NUMBER;
        }

[ \t]     ;

\n        return '\n';

"+"       return '+';
"-"       return '-';

/* Complete the code for 1. Multiplication operator, 2. Division operator, 3. Exponentiation
operator, and 4. Parentheses */

.         return yytext[0];
```

```
%%
```

```
int yywrap()
{
    return 1;
}
```

8. Partial YACC Program (parser.y)

The following parser partially implements:

- Addition +
- Subtraction -
- Parentheses ()

Students must complete the implementation for:

- Multiplication *
- Division /
- Exponentiation ^

```
%{
#include <stdio.h>
#include <stdlib.h>
```

```
void yyerror(const char *s);
int yylex();
%}
```

```
%token NUMBER
```

```
%%
```

```
input:
    input line
    |
    ;
```

```
line:
    '\n'
    | expr '\n'
    {
        printf("Result = %d\n", $1);
    }
    ;
```

```
expr:
    expr '+' term
    {
```

```

        $$ = $1 + $3;
    }
| expr '-' term
    {
        $$ = $1 - $3;
    }
| term
    {
        $$ = $1;
    }
;

```

```

term:
    factor
    {
        $$ = $1;
    }
;

```

```

factor:
    NUMBER
    {
        $$ = $1;
    }
;

```

/* Add the codes in proper place to support 1. Multiplication, 2. Division, 3. Exponentiation, 4. Parentheses and 5. Unary Minus */

```
%%
```

```

void yyerror(const char *s)
{
    printf("Syntax Error\n");
}

```

```

int main()
{
    printf("Enter Expression:\n");
    yyparse();
    return 0;
}

```

9. Tasks to Complete

Students must complete the following:

In FLEX File

- Multiplication operator *
- Division operator /
- Exponentiation operator ^
- Parentheses tokens

In YACC File

- Multiplication grammar rule
- Division grammar rule
- Exponentiation grammar rule
- Proper precedence handling

10. Compilation Steps**Step 1: Generate Parser File**

```
yacc -d parser.y
```

or

```
bison -d parser.y
```

Step 2: Generate Lexer File

```
flex lexer.l
```

Step 3: Compile the Program

```
gcc lex.yy.c y.tab.c -o parser
```

Step 4: Run the Program**Linux**

```
./parser
```

Windows

```
parser.exe
```

11. Sample Input and Output

Example 1

Input

5+3-2

Output

Result = 6

Example 2

Input

(10-3)+2

Output

Result = 9

12. Extensions

Additionally implement:

- Floating-point numbers
- Unary minus

13. Sample Viva Questions

1. What is the role of YACC in compiler design?
2. What is lexical analysis?
3. What is syntax analysis?
4. What is a context-free grammar?
5. How does FLEX communicate with YACC?
6. What is operator precedence?
7. Why is exponentiation precedence higher than multiplication?
8. What is shift-reduce parsing?

14. Report Submission

Students must submit a **single document/file** containing:

- lexer.l
- parser.y
- input.txt
- output.txt

The document should clearly include:

- Complete source code
- Sample input
- Generated output